

# GUÍA DE TESTING — NIVEL 3: AUTH

## SEGURIDAD EN SUPABASE

### Seguridad de Autenticación

## 3.1 — Tests: Configuración de Auth Seguro

Estos tests verifican las **configuraciones base de seguridad**: signup, contraseñas, rate limiting y confirmación de email.

### Test 3.1.1 — Registro exitoso con metadata

Verificar que el endpoint de signup acepta metadata y crea la cuenta correctamente.

#### Postman/Insomnia:

```
POST {{SUPABASE_URL}}/auth/v1/signup

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
{
  "email": "nuevo.test@upds.edu.bo",
  "password": "NuevoTest2026!",
  "data": {
    "nombre_completo": "Nuevo Estudiante Test",
    "rol": "estudiante"
  }
}
```

#### Examinar la respuesta JSON:

```
// Campos clave en la respuesta:
{
  "id": "uuid-nuevo-usuario",           // UUID generado por auth
  "email": "nuevo.test@upds.edu.bo",
  "user_metadata": {
    "nombre_completo": "Nuevo Estudiante Test",
    "rol": "estudiante"                // La metadata que enviamos
  },
  "access_token": "eyJ...",           // JWT para usar inmediatamente
  "refresh_token": "xxx..."
}
```

| Acción / Test                        | Rol  | Resultado esperado                  | ☑/✗ |
|--------------------------------------|------|-------------------------------------|-----|
| Signup con email y metadata          | anon | HTTP 200, retorna id + access_token | ☑   |
| Verificar user_metadata en respuesta | anon | Contiene nombre_completo y rol      | ☑   |

#### Importante:

Guardá el access\_token de la respuesta como {{JWT\_NUEVO}} y el id como {{UUID\_NUEVO}}. Los necesitarás en los siguientes tests.

### Test 3.1.2 — Validación de contraseña débil

Verificar que GoTrue rechaza contraseñas que no cumplen los requisitos mínimos.

#### Postman/Insomnia:

```
POST {{SUPABASE_URL}}/auth/v1/signup

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
{
  "email": "debil@test.upds.edu.bo",
  "password": "123"
}
```

| Acción / Test                            | Rol  | Resultado esperado            | ☑/✗ |
|------------------------------------------|------|-------------------------------|-----|
| Signup con password '123' (3 caracteres) | anon | Error 422: password too short | ✗   |
| Signup con password " (vacía)            | anon | Error 422: password required  | ✗   |

#### Configuración:

GOTRUE\_PASSWORD\_MIN\_LENGTH controla la longitud mínima.  
El valor por defecto es 6 caracteres. En el módulo configuramos 8.  
En Docker, se define en el .env. En Supabase Cloud, en el Dashboard.

### Test 3.1.3 — Login y obtención de JWT

Verificar el flujo completo de login y la estructura del JWT retornado.

#### Postman/Insomnia:

```
POST {{SUPABASE_URL}}/auth/v1/token?grant_type=password

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
{
  "email": "nuevo.test@upds.edu.bo",
  "password": "NuevoTest2026!"
}
```

#### Decodificar el JWT (usar [jwt.io](https://jwt.io) o [jwt.ms](https://jwt.ms)):

```
// Copiar el access_token y pegarlo en https://jwt.io
// El payload decodificado muestra:
{
  "sub": "uuid-del-usuario",           // auth.uid() usa este campo
  "email": "nuevo.test@upds.edu.bo",
  "role": "authenticated",           // auth.role() usa este campo
  "aud": "authenticated",
  "exp": 1745000000,                 // Expiración (epoch)
  "user_metadata": {
    "nombre_completo": "Nuevo Estudiante Test",
    "rol": "estudiante"
  }
}
```

```
}
}
```

| Acción / Test                  | Rol  | Resultado esperado                             | ☑/✗ |
|--------------------------------|------|------------------------------------------------|-----|
| Login con credenciales válidas | anon | HTTP 200, retorna access_token + refresh_token | ☑   |
| Decodificar JWT en jwt.io      | N/A  | sub = UUID, role = 'authenticated'             | ☑   |
| Verificar exp (expiración)     | N/A  | Expira en ~3600 seg (1 hora)                   | ☑   |
| Login con password incorrecto  | anon | Error 400: invalid credentials                 | ✗   |
| Login con email inexistente    | anon | Error 400: invalid credentials                 | ✗   |

**Distinción crítica:**

Observá que auth.role() retorna 'authenticated' (rol de PostgreSQL), NO el rol de tu app (estudiante/docente/admin). El rol de tu app está en user\_metadata o, si configurás custom claims, dentro del JWT. Este es un error conceptual muy común que el módulo enfatiza.

### Test 3.1.4 — Signup con email duplicado

Verificar el comportamiento cuando se intenta registrar un email que ya existe.

#### Postman/Insomnia:

```
-- Intentar registrar el mismo email que ya existe
POST {{SUPABASE_URL}}/auth/v1/signup

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
{
  "email": "nuevo.test@upds.edu.bo",
  "password": "OtraPassword123!"
}
```

| Acción / Test                  | Rol  | Resultado esperado                 | <input checked="" type="checkbox"/> /✕ |
|--------------------------------|------|------------------------------------|----------------------------------------|
| Signup con email ya registrado | anon | Error 400: user already registered | ✕                                      |

#### Nota sobre enumeración de emails:

Por seguridad, algunos proveedores retornan un HTTP 200 falso para no revelar si el email existe (protección contra enumeración). El comportamiento exacto depende de la configuración de Supabase. Verificá cómo responde tu instancia específica.

## 3.2 — Tests: Auth Hook (handle\_new\_user)

Estos tests verifican que el trigger **on\_auth\_user\_created** crea automáticamente un perfil en **usuarios\_perfil** cada vez que un usuario se registra.

### Preparación: Verificar que el Auth Hook existe

```
-- Ejecutar en SQL Editor para confirmar que el trigger está activo
SELECT
  trigger_name,
  event_manipulation,
  action_timing,
  action_statement
FROM information_schema.triggers
WHERE event_object_schema = 'auth'
  AND event_object_table = 'users';

-- Resultado esperado: on_auth_user_created, INSERT, AFTER,
-- EXECUTE FUNCTION public.handle_new_user()
```

### Test 3.2.1 — Signup crea perfil automáticamente

Verificar que al registrarse, el trigger **handle\_new\_user** inserta una fila en **usuarios\_perfil** con los datos de la metadata.

#### SQL Editor (verificar perfil del usuario creado en Test 3.1.1):

```
-- Buscar el perfil del usuario nuevo (creado en Test 3.1.1)
SELECT id, nombre_completo, rol, activo, eliminado, created_at
FROM public.usuarios_perfil
WHERE id = '{{UUID_NUEVO}}';

-- Resultado esperado:
-- nombre_completo = 'Nuevo Estudiante Test' (de la metadata)
-- rol = 'estudiante' (de la metadata)
-- activo = true (default)
-- eliminado = false (default)
```

#### Verificación alternativa desde Postman/Insomnia:

```
-- Usar el JWT del nuevo usuario para leer SU perfil
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=id,nombre_completo,rol

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_NUEVO}}
```

| Acción / Test                              | Rol        | Resultado esperado                     | ☑/✕ |
|--------------------------------------------|------------|----------------------------------------|-----|
| Verificar perfil creado por Auth Hook      | SQL Editor | Fila existe con nombre y rol correctos | ☑   |
| Nuevo usuario lee su propio perfil vía API | Nuevo user | Retorna 1 fila con sus datos           | ☑   |

### Test 3.2.2 — Signup SIN metadata: fallback a valores por defecto

Verificar que si el usuario no envía metadata, el hook usa los valores COALESCE de fallback.

#### Postman/Insomnia:

```
POST {{SUPABASE_URL}}/auth/v1/signup
```

Headers:

```
apikey: {{ANON_KEY}}
Content-Type: application/json
```

Body:

```
{
  "email": "sin.metadata@test.upds.edu.bo",
  "password": "SinMeta2026!"
}
```

### SQL Editor (verificar perfil):

```
-- Buscar el perfil del usuario sin metadata
SELECT id, nombre_completo, rol
FROM public.usuarios_perfil
WHERE id = (SELECT id FROM auth.users WHERE email = 'sin.metadata@test.upds.edu.bo');

-- Resultado esperado:
-- nombre_completo = 'sin.metadata'      (split_part del email: fallback 3)
-- rol = 'estudiante'                    (COALESCE default: fallback 2)
```

| Acción / Test                       | Rol        | Resultado esperado                   | ☑/✗ |
|-------------------------------------|------------|--------------------------------------|-----|
| Signup sin campo data en el body    | anon       | HTTP 200, cuenta creada exitosamente | ☑   |
| Verificar nombre_completo en perfil | SQL Editor | Usa parte del email antes del @      | ☑   |
| Verificar rol en perfil             | SQL Editor | Asignado 'estudiante' por defecto    | ☑   |

### Lógica COALESCE del Auth Hook:

La cadena COALESCE en handle\_new\_user funciona así:

1. Intenta: raw\_user\_meta\_data ->> 'nombre\_completo'
2. Si NULL: raw\_user\_meta\_data ->> 'full\_name' (OAuth)
3. Si NULL: split\_part(email, '@', 1) (fallback final)

Para el rol:

1. Intenta: raw\_user\_meta\_data ->> 'rol'
2. Si NULL: 'estudiante' (default seguro)

### Test 3.2.3 — Intento de inyectar rol 'admin' desde el signup

Verificar qué sucede si un usuario malintencionado envía rol: 'admin' en la metadata del signup. El Auth Hook acepta este valor, pero las capas posteriores (RLS + triggers) lo contienen.

#### Postman/Insomnia:

```
POST {{SUPABASE_URL}}/auth/v1/signup

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
{
  "email": "intruso@test.upds.edu.bo",
  "password": "Intruso2026!",
  "data": {
    "nombre_completo": "Intruso Malicioso",
    "rol": "admin"
  }
}
```

#### SQL Editor (verificar perfil):

```
SELECT id, nombre_completo, rol
FROM public.usuarios_perfil
WHERE id = (SELECT id FROM auth.users WHERE email = 'intruso@test.upds.edu.bo');

-- RESULTADO: rol = 'admin' (!!)
```

-- El Auth Hook NO valida el rol que viene en la metadata.  
 -- Esto es una VULNERABILIDAD si no hay capas adicionales.

#### Ahora verificar que las capas de defensa lo contienen:

```
-- ¿El intruso puede ver TODOS los perfiles como admin?
-- Depende de cómo está implementado es_admin()
-- Si es_admin() verifica eliminado = false AND activo = true,
-- y el Auth Hook creó el perfil con esos valores por defecto...
-- entonces SÍ podría tener acceso admin.

-- VERIFICAR: ¿Puede el intruso ver todos los perfiles?
```

#### Postman/Insomnia (probar acceso del intruso):

```
-- Obtener JWT del intruso
POST {{SUPABASE_URL}}/auth/v1/token?grant_type=password

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
{ "email": "intruso@test.upds.edu.bo", "password": "Intruso2026!" }
```

-- Luego usar su JWT para leer todos los perfiles

```
GET {{SUPABASE_URL}}/rest/v1/usuarios_perfil?select=id,nombre_completo,rol

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_INTRUSO}}
```

| Acción / Test                               | Rol        | Resultado esperado                    | ☑/✕ |
|---------------------------------------------|------------|---------------------------------------|-----|
| Signup con rol: 'admin' en metadata         | anon       | Signup exitoso (Auth no valida roles) | ☑   |
| Verificar que el perfil tiene rol = 'admin' | SQL Editor | Sí, el Auth Hook aceptó el valor      | ☑   |
| Intruso usa es_admin() vía RLS              | Intruso    | PODRÍA tener acceso admin (⚠)         | ☑   |

### ANÁLISIS DE SEGURIDAD — Hallazgo crítico:

⚠ VULNERABILIDAD DETECTADA: El Auth Hook toma el rol de la metadata sin validar. Cualquier usuario puede auto-asignarse rol 'admin' al registrarse.

MITIGACIONES (implementar al menos una):

1. FORZAR rol en el hook: ignorar metadata y siempre asignar 'estudiante'
2. VALIDAR en el hook: solo aceptar roles de una lista permitida
3. Usar un ENUM o CHECK constraint en la tabla
4. Que solo un admin pueda cambiar roles (ya cubierto por el trigger)

En producción, la Mitigación 1 es la más segura.

### Mitigación recomendada (aplicar en SQL Editor):

```
-- OPCIÓN 1: Forzar rol 'estudiante' siempre en el hook
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO public.usuarios_perfil (id, nombre_completo, rol)
    VALUES (
        NEW.id,
        COALESCE(
            NEW.raw_user_meta_data ->> 'nombre_completo',
            NEW.raw_user_meta_data ->> 'full_name',
            split_part(NEW.email, '@', 1)
        ),
        'estudiante' -- SIEMPRE estudiante, nunca de la metadata
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Corregir el intruso ya creado
UPDATE public.usuarios_perfil SET rol = 'estudiante'
WHERE id = (SELECT id FROM auth.users WHERE email = 'intruso@test.upds.edu.bo');
```



### Test 3.2.4 — Verificar que la mitigación funciona

Después de aplicar la mitigación, registrar otro usuario con rol: 'admin' y verificar que se ignora.

#### Postman/Insomnia:

```
POST {{SUPABASE_URL}}/auth/v1/signup

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
{
  "email": "intruso2@test.upds.edu.bo",
  "password": "Intruso2026!",
  "data": {
    "nombre_completo": "Segundo Intruso",
    "rol": "admin"
  }
}
```

#### SQL Editor (verificar):

```
SELECT nombre_completo, rol
FROM public.usuarios_perfil
WHERE id = (SELECT id FROM auth.users WHERE email = 'intruso2@test.upds.edu.bo');

-- Resultado esperado: rol = 'estudiante' (la metadata fue ignorada)
```

| Acción / Test                                 | Rol        | Resultado esperado                      | ☑/✕ |
|-----------------------------------------------|------------|-----------------------------------------|-----|
| Signup con rol: 'admin' (post-mitigación)     | anon       | Signup exitoso, pero rol = 'estudiante' | ☑   |
| Verificar que la metadata de rol fue ignorada | SQL Editor | rol = 'estudiante' en el perfil         | ☑   |

### 3.3 — Tests: Custom Claims en JWT

Estos tests verifican que el **custom\_access\_token\_hook** inyecta el rol de la app directamente en el JWT, y que las políticas pueden usarlo con **auth.jwt() -> 'user\_role'**.

#### Preparación: Configurar Custom Claims

Ejecutá este SQL si aún no tenés el hook configurado:

```
-- 1. Crear la función del hook
CREATE OR REPLACE FUNCTION public.custom_access_token_hook(event JSONB)
RETURNS JSONB AS $$
DECLARE
    claims JSONB;
    user_rol TEXT;
BEGIN
    SELECT rol INTO user_rol
    FROM public.usuarios_perfil
    WHERE id = (event ->> 'user_id')::UUID
    AND eliminado = false AND activo = true;

    claims := event -> 'claims';
    claims := jsonb_set(claims, '{user_role}',
        to_jsonb(COALESCE(user_rol, 'estudiante')));
    event := jsonb_set(event, '{claims}', claims);
    RETURN event;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- 2. Conceder permisos
GRANT USAGE ON SCHEMA public TO supabase_auth_admin;
GRANT EXECUTE ON FUNCTION public.custom_access_token_hook TO supabase_auth_admin;
REVOKE EXECUTE ON FUNCTION public.custom_access_token_hook FROM authenticated, anon, public;

-- 3. HABILITAR en el Dashboard:
-- Authentication → Hooks → Custom Access Token (JWT)
-- Activar y seleccionar: public.custom_access_token_hook
```

#### Activación obligatoria:

⚠ El hook solo se activa después de habilitarlo en el Dashboard.  
Sin ese paso, el JWT no incluirá el claim 'user\_role'.  
En Docker, se configura vía variables de entorno de GoTrue.

#### Test 3.3.1 — Verificar user\_role en el JWT

Hacer login con cada usuario y decodificar el JWT para verificar que contiene el claim **user\_role**.

#### Postman/Insomnia (login de cada rol):

```
-- Login como admin
POST {{SUPABASE_URL}}/auth/v1/token?grant_type=password

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
{ "email": "admin@test.upds.edu.bo", "password": "TestAdmin123!" }
```

```
-- Repetir con docente@test.upds.edu.bo y alumno@test.upds.edu.bo
-- Copiar cada access_token y decodificarlo en jwt.io
```

### JWT decodificado esperado (ejemplo admin):

```
{
  "sub": "uuid-admin",
  "email": "admin@test.upds.edu.bo",
  "role": "authenticated",
  "user_role": "admin",           // ← CUSTOM CLAIM inyectado
  "exp": 1745000000
}
```

| Acción / Test                     | Rol        | Resultado esperado                     | ☑/✕ |
|-----------------------------------|------------|----------------------------------------|-----|
| Login admin, decodificar JWT      | Admin      | user_role = 'admin' en el payload      | ☑   |
| Login docente, decodificar JWT    | Docente    | user_role = 'docente' en el payload    | ☑   |
| Login estudiante, decodificar JWT | Estudiante | user_role = 'estudiante' en el payload | ☑   |
| Campo 'role' (de PostgreSQL)      | Todos      | Siempre 'authenticated' (no cambia)    | ☑   |

### Test 3.3.2 — Política RLS usando custom claims

Verificar que una política basada en `auth.jwt()` ->> `'user_role'` funciona correctamente sin consultar `usuarios_perfil`.

#### SQL Editor (crear política de prueba):

```
-- Política que usa custom claims en lugar de consultar la tabla
CREATE POLICY "materias_crud_admin_jwt"
  ON public.materias
  FOR ALL
  TO authenticated
  USING (
    (SELECT auth.jwt() ->> 'user_role') = 'admin'
  )
  WITH CHECK (
    (SELECT auth.jwt() ->> 'user_role') = 'admin'
  );
```

#### Postman/Insomnia:

```
-- Admin intenta INSERT en materias usando la política JWT
POST {{SUPABASE_URL}}/rest/v1/materias

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ADMIN}}
  Content-Type: application/json
  Prefer: return=representation

Body:
{
  "codigo": "JWT-101",
  "nombre": "Test Custom Claims",
  "creditos": 3,
  "semestre": "2026-I"
}
```

```
-- Estudiante intenta la misma operación
POST {{SUPABASE_URL}}/rest/v1/materias

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ALUMNO}}
  Content-Type: application/json
  Prefer: return=representation

Body:
{
  "codigo": "HACK-002",
  "nombre": "Intento Estudiante",
  "creditos": 1,
  "semestre": "2026-I"
}
```

| Acción / Test                      | Rol        | Resultado esperado                   | ☑/✗ |
|------------------------------------|------------|--------------------------------------|-----|
| Admin INSERT usando política JWT   | Admin      | INSERT exitoso (user_role = 'admin') | ☑   |
| Estudiante INSERT con política JWT | Estudiante | Error 403: violates RLS policy       | ✗   |
| Docente INSERT con política JWT    | Docente    | Error 403 (user_role != 'admin')     | ✗   |

#### Limpieza:

```
-- Eliminar materia de prueba y política temporal
DELETE FROM public.materias WHERE codigo = 'JWT-101';
DROP POLICY IF EXISTS "materias_crud_admin_jwt" ON public.materias;
```

### Test 3.3.3 — Custom claim con usuario eliminado (soft delete)

Verificar que si un usuario fue soft-deleted, el hook retorna 'estudiante' como fallback porque la query filtra **eliminado = false AND activo = true**.

#### SQL Editor (simular eliminación):

```
-- Soft-delete del estudiante de prueba
UPDATE public.usuarios_perfil
SET eliminado = true, activo = false
WHERE id = '{{UUID_ALUMNO}}';
```

#### Postman/Insomnia (nuevo login del estudiante eliminado):

```
-- El estudiante aún puede loguearse (auth.users no fue tocado)
POST {{SUPABASE_URL}}/auth/v1/token?grant_type=password

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
{ "email": "alumno@test.upds.edu.bo", "password": "TestAlumno123!" }

-- Decodificar el nuevo JWT en jwt.io
-- Buscar el claim 'user_role'
```

| Acción / Test                   | Rol       | Resultado esperado                           | ☑/✕ |
|---------------------------------|-----------|----------------------------------------------|-----|
| Login del usuario eliminado     | Eliminado | Login exitoso (auth.users intacto)           | ☑   |
| Verificar user_role en JWT      | Eliminado | user_role = 'estudiante' (fallback COALESCE) | ☑   |
| Intentar leer su perfil vía API | Eliminado | Array vacío (RLS filtra eliminado = false)   | ☑   |

#### Restaurar para futuros tests:

```
UPDATE public.usuarios_perfil
SET eliminado = false, activo = true
WHERE id = '{{UUID_ALUMNO}}';
```

#### Seguridad del hook de claims:

El hook de custom claims tiene doble protección para eliminados:

1. La query filtra eliminado = false AND activo = true
2. Si no encuentra fila, COALESCE retorna 'estudiante'

Resultado: un usuario eliminado obtiene un JWT con el mínimo privilegio.  
No puede escalar privilegios porque nunca recibe su rol original.

### 3.4 — Tests: Validación de Sesión y Tokens

Estos tests verifican el **ciclo de vida del JWT**: expiración, refresh, y la diferencia entre `getUser()` y `getSession()`.

#### Test 3.4.1 — `getUser()` vs `getSession()`: diferencia de seguridad

El módulo enfatiza que `getUser()` **verifica el JWT contra el servidor**, mientras que `getSession()` **solo lee la cookie localmente**. Podemos simular esto con Postman.

##### Test A — `getUser()` (verificación server-side):

```
-- Equivalente a supabase.auth.getUser()
GET {{SUPABASE_URL}}/auth/v1/user

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ALUMNO}}
```

##### Test B — Request con JWT expirado o manipulado:

```
-- Usar un JWT alterado manualmente (cambiar un carácter)
GET {{SUPABASE_URL}}/auth/v1/user

Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.MANIPULADO
```

| Acción / Test                                   | Rol        | Resultado esperado                  | ☑/✗ |
|-------------------------------------------------|------------|-------------------------------------|-----|
| <code>getUser()</code> con JWT válido           | Estudiante | HTTP 200, retorna datos del usuario | ☑   |
| <code>getUser()</code> con JWT manipulado       | N/A        | Error 401: invalid token            | ✗   |
| <code>getUser()</code> sin Authorization header | anon       | Error 401: not authenticated        | ✗   |

##### Regla de seguridad crítica:

`getUser()` hace una verificación real contra GoTrue (el servidor de auth).  
`getSession()` solo decodifica el JWT localmente sin verificar firma.

En Next.js Server Components, siempre usar `getUser()`.  
`getSession()` es aceptable en el cliente para UI rápida, pero **NUNCA** para decisiones de seguridad (autorización, acceso a datos sensibles).

#### Test 3.4.2 — Refresh token

Verificar que el `refresh_token` permite obtener un nuevo `access_token` sin re-autenticarse.

##### Postman/Insomnia:

```
POST {{SUPABASE_URL}}/auth/v1/token?grant_type=refresh_token

Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json

Body:
```

```
{
  "refresh_token": "{{REFRESH_TOKEN_ALUMNO}}"
}
```

| Acción / Test                                    | Rol        | Resultado esperado                           | ☑/✗ |
|--------------------------------------------------|------------|----------------------------------------------|-----|
| Refresh con token válido                         | Estudiante | HTTP 200, nuevo access_token + refresh_token | ☑   |
| Refresh con token inválido/expirado              | N/A        | Error 400: invalid refresh token             | ✗   |
| Verificar que el viejo access_token aún funciona | Estudiante | Funciona hasta que expire (no se invalida)   | ☑   |

### Test 3.4.3 — Logout (invalidar sesión)

Verificar que después de logout, el refresh\_token ya no funciona.

#### Postman/Insomnia:

```
-- Cerrar sesión
POST {{SUPABASE_URL}}/auth/v1/logout
```

```
Headers:
  apikey: {{ANON_KEY}}
  Authorization: Bearer {{JWT_ALUMNO}}
```

```
-- Intentar refresh después del logout
POST {{SUPABASE_URL}}/auth/v1/token?grant_type=refresh_token
```

```
Headers:
  apikey: {{ANON_KEY}}
  Content-Type: application/json
```

```
Body:
{ "refresh_token": "{{REFRESH_TOKEN_ALUMNO}}" }
```

| Acción / Test                               | Rol        | Resultado esperado                      | ☑/✗ |
|---------------------------------------------|------------|-----------------------------------------|-----|
| Logout del estudiante                       | Estudiante | HTTP 204 (sesión cerrada)               | ☑   |
| Refresh post-logout                         | N/A        | Error 400: invalid refresh token        | ✗   |
| Access token post-logout (antes de expirar) | Estudiante | Aún funciona hasta exp (limitación JWT) | ☑   |

#### Limitación de los JWT:

Los JWT son stateless: una vez emitidos, son válidos hasta que expiran.  
 El logout invalida el refresh\_token, pero NO el access\_token actual.  
 Por eso el módulo recomienda GOTRUE\_JWT\_EXP = 3600 (1 hora):  
 un balance entre seguridad (token corto) y UX (menos re-logins).

Para invalidación inmediata, se necesitaría una blacklist de tokens, lo cual Supabase no soporta nativamente.

## Limpieza de Datos de Prueba

Ejecutá estas queries en el SQL Editor para limpiar todos los usuarios de prueba creados durante esta guía:

```
-- Eliminar perfiles de prueba del Nivel 3
DELETE FROM public.usuarios_perfil
WHERE id IN (
    SELECT id FROM auth.users
    WHERE email IN (
        'nuevo.test@upds.edu.bo',
        'sin.metadata@test.upds.edu.bo',
        'intruso@test.upds.edu.bo',
        'intruso2@test.upds.edu.bo'
    )
);

-- Eliminar cuentas auth de prueba
-- ⚠ Solo ejecutar si estos son usuarios de prueba
DELETE FROM auth.users
WHERE email IN (
    'nuevo.test@upds.edu.bo',
    'sin.metadata@test.upds.edu.bo',
    'intruso@test.upds.edu.bo',
    'intruso2@test.upds.edu.bo'
);
```



## Checklist de Verificación — Nivel 3 Completo

Marcá cada test a medida que lo completés:

| <input checked="" type="checkbox"/> | Test         | Descripción                                                  | OK |
|-------------------------------------|--------------|--------------------------------------------------------------|----|
| <input type="checkbox"/>            | <b>3.1.1</b> | Registro exitoso con metadata de nombre y rol                |    |
| <input type="checkbox"/>            | <b>3.1.2</b> | Contraseña débil rechazada por GoTrue                        |    |
| <input type="checkbox"/>            | <b>3.1.3</b> | Login exitoso, estructura del JWT, credenciales inválidas    |    |
| <input type="checkbox"/>            | <b>3.1.4</b> | Signup con email duplicado rechazado                         |    |
| <input type="checkbox"/>            | <b>3.2.1</b> | Auth Hook crea perfil automáticamente al signup              |    |
| <input type="checkbox"/>            | <b>3.2.2</b> | Signup sin metadata usa fallbacks COALESCE                   |    |
| <input type="checkbox"/>            | <b>3.2.3</b> | Inyección de rol admin detectada + mitigación aplicada       |    |
| <input type="checkbox"/>            | <b>3.2.4</b> | Post-mitigación: rol siempre 'estudiante' al registrarse     |    |
| <input type="checkbox"/>            | <b>3.3.1</b> | Custom claim user_role presente en JWT de cada rol           |    |
| <input type="checkbox"/>            | <b>3.3.2</b> | Política RLS basada en auth.jwt() ->> 'user_role' funciona   |    |
| <input type="checkbox"/>            | <b>3.3.3</b> | Usuario eliminado recibe user_role = 'estudiante' (fallback) |    |
| <input type="checkbox"/>            | <b>3.4.1</b> | getUser() vs token manipulado: validación server-side        |    |
| <input type="checkbox"/>            | <b>3.4.2</b> | Refresh token renueva access_token correctamente             |    |
| <input type="checkbox"/>            | <b>3.4.3</b> | Logout invalida refresh, access sigue hasta exp              |    |